

Documentation Submission

Synthetic customer-support ticket triage across Chat and IDE UX types

Student	Cyusa Kheven	Lab	Multi-Stage AI Workflow
Workflow	ChatGPT to Codex in VS Code	Verified	21 July 2026
Dataset	10 synthetic support tickets	Final output	Prioritized Markdown action report

Submission summary. This lab implements a real two-stage workflow. ChatGPT classifies a synthetic ticket dataset into a strict JSON handoff. The Codex IDE stage consumes that validated handoff and produces a deterministic, prioritized action report. The implementation includes schema validation, cross-dataset semantic checks, automated tests, and authentic evidence from both UX surfaces.

Deliverables at a glance

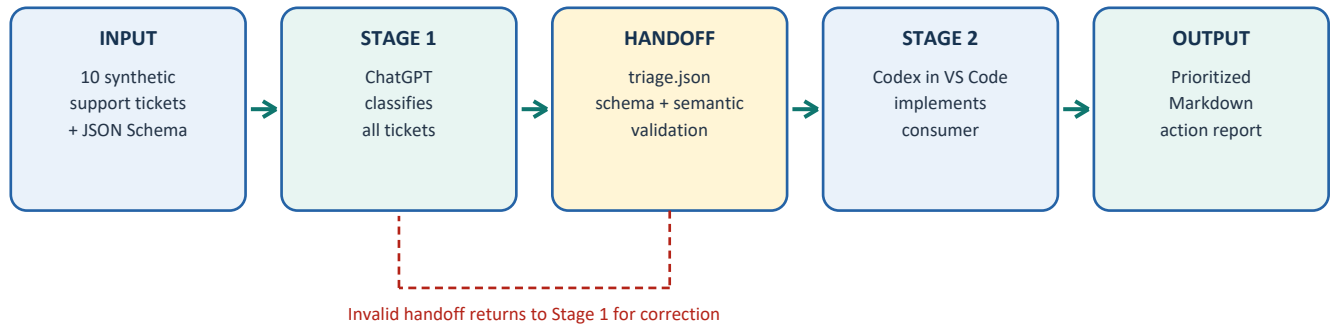
Deliverable	Status	Primary evidence
Cross-UX workflow	Complete	ChatGPT handoff consumed by Codex implementation
Workflow diagram	Complete	Included on page 2
Written documentation	Complete	Steps, testing, adaptability, efficiency, and evidence
Automated verification	10 passed, 0 failed	evidence/test-results.txt

All data in this project is explicitly synthetic. No real customer information is used.

ARCHITECTURE AND HANDOFF

Workflow overview

The work is split at a deliberate contract boundary. Stage 1 performs AI-assisted classification; Stage 2 validates and deterministically transforms those classifications without silently changing them.



The JSON handoff is the explicit boundary between the chat UX and IDE UX.

Handoff contract and safeguards

Control	Implementation	Risk addressed
Strict schema	JSON Schema Draft 2020-12 with <code>additionalProperties: false</code>	Unexpected or malformed fields
Ticket identity	IDs must match <code>^TKT-[0-9]{3}\$</code>	Invalid identifier shapes
Dataset reconciliation	Dataset ID and source ticket IDs must match	Missing, duplicated, or invented tickets
Clarification consistency	Flag and information-gap list are checked together	Contradictory triage decisions
Failure behavior	Invalid input exits non-zero and produces no report	Bad data propagating to the output

Separation of responsibility

ChatGPT supplies the classifications. The JavaScript consumer validates, counts, sorts, and formats those decisions. This makes it possible to identify whether an issue originated in classification, contract validation, or report generation.

REPRODUCIBLE PROCESS

End-to-end workflow steps

Stage 1 - Chat UX

1. Start a new ChatGPT conversation to avoid unrelated context.
2. Attach `data/raw-tickets.json` and `schemas/triage.schema.json`.
3. Send the reusable prompt in `prompts/stage-1-chat.md`.
4. Receive one structured JSON object covering all 10 tickets.
5. Save the response unchanged as `handoff/triage.json`.
6. Run `npm run validate:stage1` before handoff.
7. Retain the authentic input and output screenshots as evidence.

Stage 1 integrity rule: classifications were not manually edited. A failed validation would return the errors to Stage 1 instead of silently repairing the handoff.

Stage 2 - IDE UX

1. Open the repository in Visual Studio Code and confirm the validated handoff is present.
2. Open the Codex IDE extension and send `prompts/stage-2-codex-ide.md`.
3. Review the implementation and the nine-file change set.
4. Run `npm test` and confirm all 10 tests pass.
5. Run `npm run stage2` to validate the handoff and generate the report.
6. Review `output/action-report.md` and retain the IDE evidence screenshots.

Commands used

Command	Purpose	Expected result
<code>npm install</code>	Install the pinned Ajv dependency	Dependencies available
<code>npm run validate:stage1</code>	Validate the ChatGPT handoff	10 tickets validated
<code>npm test</code>	Run positive and negative tests	10 passed, 0 failed
<code>npm run stage2</code>	Validate and generate the final report	Exit 0 and report written

QUALITY AND HALLUCINATION DEFENCE

Validation and testing

Node.js's built-in test runner covers both the successful workflow and each important failure mode. The verified run on 21 July 2026 used Node.js v24.14.0 and npm 11.9.0.

Test case	Expected behavior	Result
Valid handoff	Generate the complete report deterministically	Passed
Missing required property	Reject with a field-specific schema error	Passed
Missing source ticket ID	Reject	Passed
Duplicate ticket ID	Reject	Passed
Invented ticket ID	Reject	Passed
Dataset ID mismatch	Reject	Passed
Clarification true, no gaps	Reject inconsistent handoff	Passed
Clarification false, gaps present	Reject inconsistent handoff	Passed
Empty ticket list	Reject under the schema	Passed
Invalid CLI input	Exit non-zero and write no report	Passed

Recorded execution evidence

Verified command	Exit	Observed result
npm.cmd run validate:stage1	0	10 tickets passed schema and semantic validation
npm.cmd test	0	10 passed, 0 failed
npm.cmd run stage2	0	Action report regenerated

The exact terminal output is retained in `evidence/test-results.txt`; corresponding test and successful-run screenshots are indexed under `evidence/screenshots/`.

PRIORITIZED ACTION REPORT

Generated result

The final report contains all 10 validated tickets, ordered by priority, with counts by priority and category. It formats the Stage 1 classifications without reclassifying them.

PRIORITY COUNTS		CATEGORY COUNTS	
Critical	1	Billing	3
High	3	Feature request	2
Medium	3	Technical	2
Low	3	Account access / outage / other	1 each

Key action outcomes

Ticket	Priority	Outcome
TKT-005	Critical	Escalated as a production incident: all 35 agents receive 503 errors after sign-in.
TKT-001 / 002 / 008	High	Duplicate billing, account access, and post-cancellation billing are routed ahead of normal work.
TKT-010	Medium	Isolated for clarification because the affected feature, symptoms, error, reproduction steps, and environment are missing.

Report behavior verified

- Identical input produces identical output.
- The report identifies dataset version `synthetic-support-v1` and workflow version `1.0`.
- Tickets are ordered critical, high, medium, then low.
- Priority and category totals are calculated automatically.
- Clarification needs are extracted into a dedicated section.

Generated artifact: `output/action-report.md`

EVALUATION CRITERIA

Adaptability and efficiency

Adaptability

The handoff contract is provider-neutral. The workflow uses plain JSON, JSON Schema Draft 2020-12, Markdown prompts, ordinary JavaScript commands, and Markdown output. The deterministic tests do not call an AI model.

Component	Provider-neutral design	Substitution path
Stage 1	Receives a dataset, schema, and Markdown prompt	Another chat AI can be used if it returns conforming JSON
Handoff	No model name, proprietary message format, or provider metadata	Any producer or consumer can implement the same schema
Stage 2	JavaScript, Ajv, and Node's built-in test runner	Another IDE assistant can be used if it passes the same tests
Output	Standard Markdown	HTML or CSV requires changing only the formatter and tests

Efficiency

After ChatGPT produces classifications, one `npm run stage2` command performs the repeatable work below for the full 10-ticket dataset:

- Validate every ticket against the schema.
 - Reconcile all source and handoff ticket IDs.
 - Detect missing, duplicate, and invented ticket IDs.
- Check clarification flags against information-gap lists.
 - Calculate four priority totals and six category totals.
 - Sort tickets, extract clarification needs, and format the final report.

Measurement boundary: no controlled manual baseline was recorded, so this submission does not claim a specific number of minutes or percentage saved. The efficiency claim is limited to the repeatable manual checks, calculations, sorting, and formatting that the implemented command demonstrably automates.

REVIEWER REFERENCE

Evidence and rubric alignment

Criterion	Weight	Implementation and evidence
Functionality	40%	Validated handoff, 10 passing tests, successful Stage 2 run, and generated action report
Adaptability	20%	Provider-neutral schema, reusable prompts, plain JavaScript, and no AI SDK dependency
Documentation	20%	This consolidated PDF includes the diagram, steps, testing, evidence, and design rationale
Efficiency	20%	One command automates validation, reconciliation, counting, sorting, clarification extraction, and formatting

Evidence index

Artifact	What it demonstrates
evidence/screenshots/stage-1-input.png	ChatGPT input files and Stage 1 prompt
evidence/screenshots/stage-1-output.png	Structured JSON returned by ChatGPT
evidence/screenshots/stage-2-completion-summary.png	Codex IDE completion summary
evidence/screenshots/stage-2-diff.png	Reviewed nine-file implementation change set
evidence/screenshots/vs-code-complete-tests.png	10 passing and 0 failing tests
evidence/screenshots/npm-run-stage-2-success.png	Successful Stage 2 command and generated report
evidence/test-results.txt	Exact retained terminal output
output/action-report.md	Final prioritized customer-support action report

Evidence integrity statement. The screenshots and command results listed above come from the actual lab run. No AI transcripts, screenshots, timings, or test results were fabricated. The Stage 1 output was saved without manual classification edits.

Repository: github.com/khevin2/mod3lab
Primary source documentation: SUBMISSION.md, README.md, and docs/